ESPE	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	CIENCIAS EXACTAS	CARRERA:	TECNOLOGIAS DE LA INFORMACION		
ASIGNATURA:	METODOS NUMERICOS	NIVEL:	3	FECHA:	22/06/2026
DOCENTE:	RODRIGO VLADIMIR BASTIDAS CHALAN	PRÁCTICA N°:	3	CALIFICACIÓN:	

Solución de ecuaciones no lineales

Catota Jefferson, Cotera Erick, Guallo Iker, Demera Darlin, Vera Jeffry

RESUMEN

El presente laboratorio expone el proceso de desarrollo e implementación de una Interfaz Gráfica de Usuario (GUI) en Scilab, diseñada para resolver sistemas de ecuaciones lineales mediante el método de descomposición LU. La aplicación permite al usuario definir de manera dinámica la dimensión del sistema, ingresar los coeficientes de la matriz A y el vector de términos independientes b, y ejecutar el algoritmo de factorización junto con las sustituciones progresiva y regresiva para obtener la solución. Durante la programación se optó por utilizar exclusivamente la estructura iterativa while en todos los bucles, lo que aseguró un control uniforme y didáctico del flujo del programa. Además, la GUI ofrece un panel de traza que muestra en tiempo real los pasos intermedios como el cálculo de las matrices L y U, y las sustituciones realizadas, y finalmente presenta las soluciones junto con las matrices resultantes de la factorización.

Palabras Claves: Descomposición LU, Scilab, Sistemas lineales.

1. INTRODUCCIÓN:

En ingeniería y ciencias aplicadas, resolver sistemas de ecuaciones lineales de la forma $Ax = b$ es un desafío central que aparece en áreas tan diversas como el análisis estructural, la dinámica de fluidos, la electrónica, la economía o la optimización. En la práctica, estos sistemas suelen involucrar un número elevado de incógnitas y coeficientes, lo que hace poco viable recurrir a métodos analíticos directos como la regla de Cramer debido a la complejidad computacional que implican (Gomila, 2025). Entre los algoritmos de factorización más utilizados se encuentra la Descomposición LU, que expresa una matriz A como el producto de una matriz triangular inferior L y una matriz triangular superior U, esta técnica permite resolver el sistema original mediante dos pasos de sustitución sucesivos progresiva y regresiva.

El presente laboratorio busca fortalecer el uso del software científico Scilab mediante el desarrollo de una interfaz gráfica interactiva que permita ejecutar, paso a paso, la descomposición LU y mostrar al usuario cada detalle del proceso. El programa fue diseñado con un enfoque didáctico y funcional, empleando únicamente bucles while para todas las operaciones iterativas.

2. OBJETIVO(S):

2.1 OBJETIVO GENERAL:

Diseñar e implementar un programa con entorno gráfico en Scilab para la resolución de sistemas de ecuaciones lineales mediante el Método de Descomposición LU, garantizando una experiencia interactiva, segura y educativa para el usuario, mostrando todos los pasos intermedios del algoritmo.

2.2 OBJETIVOS ESPECIFICOS:

- Implementar una GUI dinámica que permita al usuario definir la dimensión n del sistema, ingresar los coeficientes de la matriz A y el vector b a través de campos de edición creados en tiempo real
- Desarrollar el algoritmo de factorización LU utilizando únicamente estructuras de control while, calculando las matrices L y U según el método.
- Programar las sustituciones progresiva y regresiva para obtener la solución del sistema, mostrando en la consola de la GUI cada paso intermedio.

3. MARCO TEÓRICO:

La factorización o descomposición LU es una técnica utilizada para resolver sistemas de ecuaciones lineales de la forma $Ax = b$, donde A corresponde a la matriz de coeficientes, x representa las incógnitas del sistema y b contiene los términos independiente, este método consiste en descomponer la matriz A en dos matrices más simples, lo que facilita el proceso de encontrar la solución. A diferencia de los métodos iterativos, que requieren aproximaciones sucesivas, la descomposición LU permite obtener la solución mediante una cantidad finita de operaciones matemáticas, alcanzando un resultado exacto siempre que no se consideren los pequeños errores de redondeo propios de los cálculos realizados por una computadora (Manuel, 2014).

Una vez que las matrices L y U han sido calculadas con éxito, el sistema original $Ax = b$ se reescribe como:

$$(L * U)x = b \rightarrow L * (Ux) = b$$

Para obtener el vector solución x, el procedimiento se desarrolla en dos etapas consecutivas que aprovechan la forma triangular de las matrices obtenidas en la descomposición LU.

Primera fase
Sustitución progresiva $L * y = b$

En esta etapa se introduce un vector auxiliar y. Como la matriz L es triangular inferior y todos los elementos de su diagonal principal son iguales a uno, el sistema puede resolverse de manera sencilla siguiendo un orden secuencial. El cálculo comienza con la primera variable y_1 y continúa hasta llegar a la última y_n, utilizando los valores ya obtenidos en los pasos anteriores gracias a esta estructura, cada incógnita se determina directamente sin necesidad de realizar operaciones complejas adicionales (Chasnov, 2022).

$$y_1 = b_1 - \sum_{k=1}^{i-1} l_{ik} y_k$$

Segunda fase
Sustitución regresiva $U * x = y$

Una vez que se ha calculado por completo el vector auxiliar y, se procede a resolver el sistema asociado a la matriz U, que tiene una estructura triangular superior. En esta etapa se obtienen las incógnitas definitivas del sistema original. Debido a la forma de la matriz, el proceso se realiza en sentido inverso: se comienza calculando la última incógnita x_n y, utilizando ese resultado, se continúa de manera ascendente hasta determinar la primera x_1. Cada valor encontrado sirve para calcular los siguientes, lo que permite obtener la solución final de forma ordenada y eficiente.

$$x_1 = \frac{y_i - \sum_{k-=i+1}^{n} u_{ik} x_k}{u_{ii}}$$

En computación científica, es fundamental comprobar la validez de la solución obtenida frente a las limitaciones de precisión de la máquina. El residuo se define como el vector de discrepancia:

$$r = A * x - b$$

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

Para la implementación de la solución computacional, el código en Scilab fue desarrollado siguiendo una estructura modular, organizando el programa en diferentes funciones encargadas de tareas específicas como la gestión de la interfaz gráfica, el procesamiento de datos ingresados por el usuario y la ejecución de los cálculos matemáticos. Esta

metodología facilita el mantenimiento del código, mejora su legibilidad y permite separar claramente la lógica del método de factorización LU de los elementos visuales de interacción, logrando así una aplicación más estable y ordenada.

El primer módulo corresponde a la función **main_gui(),** encargada de inicializar la interfaz gráfica del programa. En esta sección se diseñó un entorno interactivo utilizando una temática visual tipo "Dark Matrix", pensada para ofrecer una apariencia moderna y disminuir la fatiga visual durante el uso prolongado. Mediante componentes uicontrol, se incorporaron los botones principales del sistema, tales como Generar Malla, Volcar Ejemplo, Calcular y Resetear. Además, se añadieron dos cuadros de lista **(listbox)** destinados a mostrar tanto el procedimiento realizado por el algoritmo como los resultados obtenidos al finalizar los cálculos.

El segundo módulo está representado por la función **crear_sistema_func()**, cuya responsabilidad es generar dinámicamente la estructura de entrada de datos. Cuando el usuario especifica la dimensión N del sistema de ecuaciones, limitada entre 1 y 10 para mantener una correcta visualización de la interfaz, el programa crea automáticamente una matriz de campos de texto editables. Estos campos permiten ingresar los coeficientes de la matriz A, mientras que una columna adicional se reserva para los valores del vector de términos independientes b.

Posteriormente, al presionar el botón Calcular, se ejecuta la función **resolver_sistema_func()**. En esta etapa, el programa recopila toda la información ingresada en la interfaz y convierte los valores almacenados como texto a datos numéricos mediante la función **evstr().** Una vez realizada la conversión, los datos se organizan en estructuras adecuadas y se preparan para ser enviados al módulo encargado de realizar el procesamiento matemático.

El componente principal del sistema es la función **metodo_lu_completo(),** donde se implementa el algoritmo de factorización LU. Siguiendo los requerimientos establecidos para la práctica, este módulo fue desarrollado evitando el uso de ciclos for y empleando exclusivamente estructuras de repetición while, incluso en los procesos iterativos más complejos. De esta manera, todas las operaciones necesarias para la descomposición de la matriz y la obtención de la solución se ejecutan mediante bucles while anidados, respetando las restricciones planteadas y manteniendo el correcto funcionamiento del método.

5. ANÁLISIS DE RESULTADOS:

Con el propósito de verificar el correcto funcionamiento del algoritmo y evaluar su estabilidad numérica, se realizó una prueba utilizando el caso de estudio incorporado en la función de carga automática de datos del programa. Esta prueba permitió comprobar tanto la precisión de los cálculos como la adecuada interacción entre la interfaz gráfica y el módulo matemático encargado de la factorización LU.

Para ello, se configuró un sistema de ecuaciones lineales de dimensión 3×3, empleando una matriz de coeficientes A y un vector de términos independientes b previamente definidos dentro de la aplicación.

$$A = \begin{bmatrix} 2 & 3 & -1 \\ 4 & 4 & -3 \\ 2 & -3 & 1 \end{bmatrix}, B = \begin{bmatrix} 5 \\ 3 \\ -1 \end{bmatrix}$$

Al ejecutar el comando de cálculo, la consola de traza iterativa registró el fraccionamiento de la matriz original en sus componentes triangulares bajo la restricción de diagonal de unos en la matriz inferior las matrices resultantes extraídas por el sistema fueron:

Matriz Inferior L

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 3 & 1 \end{bmatrix}$$

Matriz Superior U:

$$U = \begin{bmatrix} 2 & 3 & -1 \\ 0 & -2 & -1 \\ 0 & 0 & 5 \end{bmatrix}$$

Una vez establecidas las matrices, el panel de control iterativo procedió a mostrar los cálculos de las dos fases de sustitución. Primero, al resolver L * y = b, se obtuvo de manera transitoria el vector y = (5, -7, 15)^T. Seguidamente, mediante la sustitución regresiva U * x = y, el programa compiló y arrojó los valores exactos para el vector variable X:

- Incógnita X_1 = 1.000000
- Incógnita X_2 = 2.000000
- Incógnita X_3 = 3.000000

Para certificar la fiabilidad de estos resultados, el sistema evaluó la Precisión Residual Global calculando la norma del infinito del vector residuo (E_max =max|Ax - b|). El valor reportado por el compilador en la consola gráfica fue de 0.00e+00 (o en el orden de 10^-16 según la arquitectura de punto flotante de Scilab)

6. GRÁFICOS O FOTOGRAFÍAS:

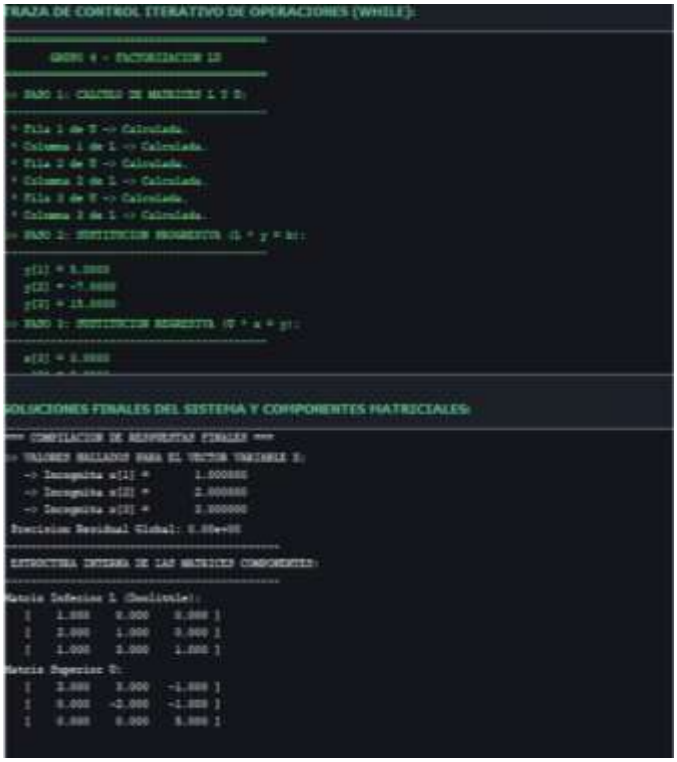
En la **Figura 1** se presenta la captura de la interfaz gráfica principal desarrollada en Scilab, diseñada bajo el esquema visual oscuro solicitado para el proyecto. En esta imagen se evidencia el funcionamiento de la estructuración dinámica de la entrada de datos.
Se puede observar que, tras definir una Dimensión del Sistema (N) de valor 3 y accionar la carga de datos (mediante la funcionalidad de volcar ejemplo), el programa renderizó y distribuyó correctamente los campos de texto editables. El panel izquierdo muestra las celdas cargadas con la Matriz de Coeficientes (A) y el Vector de términos independientes (b), conteniendo los valores exactos del caso de estudio experimental planteado en el análisis de resultados

Figura 1 : Interfaz para el usuario



En la **Figura 2** se detalla el comportamiento de los paneles derechos de la interfaz, correspondientes a las consolas de salida de datos tras presionar el botón de cálculo. En el panel superior, denominado "TRAZA DE CONTROL ITERATIVO DE OPERACIONES (WHILE)", se evidencia el registro secuencial del algoritmo. El listbox muestra el éxito del "Paso 1" al iterar y calcular cada fila y columna de las matrices L y U sin interrupciones por pivotes nulos. A continuación, documenta el "Paso 2" (Sustitución Progresiva), arrojando el vector intermedio y con los valores 5.0000, -7.0000 y 15.0000; y finalmente inicia el "Paso 3" (Sustitución Regresiva) calculando desde x_3 hacia arriba

Figura 2: Ilustración de los cálculos correspondiente



7. DISCUSIÓN:

Los resultados obtenidos durante las pruebas realizadas evidencian que la implementación del Método de Descomposición LU en Scilab funciona de manera eficiente y precisa para la resolución de sistemas de ecuaciones lineales. Al compararlo con métodos iterativos como Gauss-Seidel, se observó una ventaja importante: la solución se obtiene mediante un proceso directo, sin necesidad de establecer tolerancias de convergencia ni proporcionar valores iniciales para las incógnitas. Gracias a ello, fue posible calcular las matrices L y U de forma inmediata y, posteriormente, determinar el vector solución mediante las etapas de sustitución progresiva y regresiva con un error residual prácticamente despreciable. Estos resultados confirman la confiabilidad y estabilidad numérica de la variante de Doolittle utilizada en el desarrollo.

Un aspecto destacado del proyecto fue la implementación del algoritmo utilizando exclusivamente estructuras de repetición while para el control de las iteraciones. Esta restricción exigió una administración más cuidadosa de los índices y condiciones de ejecución en comparación con los tradicionales ciclos for. Aunque esta decisión incrementó

la complejidad del código, también permitió desarrollar una solución robusta y adaptable, integrándose correctamente con la interfaz gráfica y con el procesamiento dinámico de los datos ingresados por el usuario.

Asimismo, durante el análisis del método se identificó una limitación inherente a la descomposición LU sin pivoteo: la posibilidad de encontrar pivotes nulos o extremadamente pequeños durante el proceso de factorización, debido a que la implementación no realiza intercambios automáticos de filas, el correcto funcionamiento depende en gran medida de las características de la matriz de entrada.

8. CONCLUSIONES:

.

- Para el sistema lineal evaluado, el método de Descomposición LU demostró una alta eficiencia computacional al resolver las incógnitas en un único ciclo analítico directo, lo que valida la efectividad del algoritmo de frente a sistemas matriciales reales simples.

- La implementación de una malla dinámica de celdas mediante herramientas visuales en conjunto con la función **evstr** simplificó el flujo del programa, ya que permitió centralizar la recolección de parámetros en un solo paso y asegurar la conversión limpia de las cadenas de texto a variables numéricas antes de iniciar el procesamiento.

- Al no requerir aproximaciones iterativas, valores semilla o un desarrollo matemático analítico previo para cada ejecución (a diferencia de otros algoritmos numéricos), la factorización demostró una gran versatilidad práctica para la resolución de sistemas de ecuaciones de forma directa.

- La configuración de un criterio de parada condicional basado en la detección de pivotes nulos o críticamente cercanos a cero dentro de los bucles while anidados demostró ser el mecanismo de control más adecuado, ya que interrumpió el flujo de forma exacta para evitar ciclos de cómputo innecesarios o indeterminaciones en la máquina

- Al verificar la precisión global del sistema a través del cálculo del residuo matemático (A * x - b), se constató que el error máximo absoluto reportado en la consola fue de 0.00e+00, lo que valida experimentalmente la exactitud de las sustituciones algebraicas y demuestra la ausencia de errores significativos por redondeo en el entorno numérico de Scilab.

9. RECOMENDACIONES

- Se recomienda a los usuarios verificar que todos los campos dinámicos de la matriz de coeficientes y el vector resultante estén llenos con valores numéricos válidos antes de accionar el botón de cálculo. Dado que la interfaz emplea la función evstr() para la lectura de la malla gráfica, ingresar caracteres no reconocidos o dejar celdas completamente vacías podría generar errores de sintaxis en el compilador de Scilab
- Debido a que el algoritmo implementado es directo y no incluye una rutina de pivoteo parcial (intercambio automático de filas), se sugiere organizar el sistema de ecuaciones previamente para evitar tener ceros o valores extremadamente pequeños en la diagonal principal de la matriz original.

10. BIBLIOGRAFÍA:

Chasnov, J. (30 de Octubre de 2022). *LibreTexts*. Obtenido de https://espanol.libretexts.org/Bookshelves/Matematicas/Ecuaciones_diferenciales/%C3%81lgebra_Lineal_Aplicada_y_Ecuaciones_Diferenciales_(Chasnov)/02%3A_I._%C3%81lgebra_Lineal/02%3A_Sistemas_de_Ecuaciones_Lineales/2.05%3A_Descomposici%C3%B3n_de_LU

Gomila, J. G. (21 de Diciembre de 2025). *FroGames*. Obtenido de https://cursos.frogamesformacion.com/pages/blog/25-083-sistemas-de-ecuaciones?srsltid=AfmBOorH63FhreVI9QsqFcOJLdNbw5948dDa6Gc3HTukyfo4zkLUU3j8

Manuel, J. (5 de Mayo de 2014). *WordPress*. Obtenido de https://esimecuanalisisnumerico.wordpress.com/2014/05/05/metodo-de-lu/